

# Problemas de camino mínimo

Programación III - UNGS

# El problema de camino mínimo

---

- **Datos de entrada:**

# El problema de camino mínimo

---

- **Datos de entrada:**

1. Un grafo dirigido  $G = (N, A)$ .

# El problema de camino mínimo

---

- **Datos de entrada:**

1. Un grafo dirigido  $G = (N, A)$ .
2. Nodos  $s, t \in N$  de origen y destino.

# El problema de camino mínimo

---

- **Datos de entrada:**

1. Un grafo dirigido  $G = (N, A)$ .
2. Nodos  $s, t \in N$  de origen y destino.
3. Una función de distancia  $d : A \rightarrow \mathbb{R}_+$  asociada con los arcos.

# El problema de camino mínimo

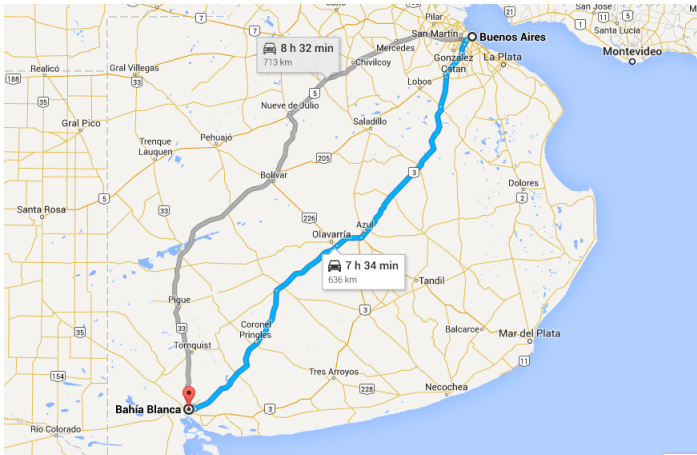
---

- **Datos de entrada:**

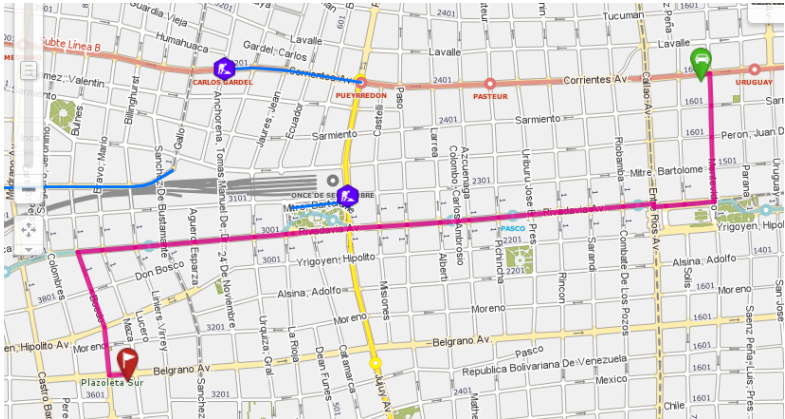
1. Un grafo dirigido  $G = (N, A)$ .
2. Nodos  $s, t \in N$  de origen y destino.
3. Una función de distancia  $d : A \rightarrow \mathbb{R}_+$  asociada con los arcos.

- **Problema:** Determinar el camino más corto entre los nodos  $s$  (origen) y  $t$  (destino).

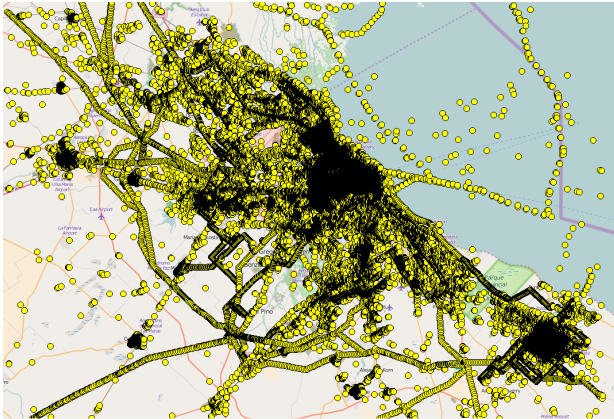
# Camino mínimo en grafos



# Camino mínimo en grafos



# Camino mínimo en grafos



# Camino mínimo en grafos

---

- Dado un grafo dirigido  $G$ , se pueden definir tres variantes de problemas sobre caminos mínimos:
  1. **Único origen - único destino:** Determinar un camino mínimo entre dos vértices específicos,  $s$  y  $t$ .
  2. **Único origen - múltiples destinos:** Determinar un camino mínimo desde un vértice específico  $s$  al resto de los vértices de  $G$ .
  3. **Múltiples orígenes - múltiples destinos:** Determinar un camino mínimo entre todo par de vértices de  $G$ .

## Camino mínimo en grafos

---

- **Arcos con peso negativo:** Si el grafo  $G$  no contiene ciclos de peso negativo o contiene alguno pero no es alcanzable desde  $s$ , entonces el problema sigue estando bien definido, aunque algunos caminos puedan tener longitud negativa.

## Camino mínimo en grafos

---

- **Arcos con peso negativo:** Si el grafo  $G$  no contiene ciclos de peso negativo o contiene alguno pero no es alcanzable desde  $s$ , entonces el problema sigue estando bien definido, aunque algunos caminos puedan tener longitud negativa.
- Sin embargo, si  $G$  tiene algún ciclo con peso negativo alcanzable desde  $s$ , el concepto de camino de peso mínimo deja de estar bien definido.

## Camino mínimo en grafos

---

- **Arcos con peso negativo:** Si el grafo  $G$  no contiene ciclos de peso negativo o contiene alguno pero no es alcanzable desde  $s$ , entonces el problema sigue estando bien definido, aunque algunos caminos puedan tener longitud negativa.
- Sin embargo, si  $G$  tiene algún ciclo con peso negativo alcanzable desde  $s$ , el concepto de camino de peso mínimo deja de estar bien definido.
- **Propiedad de subestructura óptima de un camino mínimo:**  
Sea  $P_{ij}$  un camino mínimo entre  $i$  y  $j$ , y sea  $k \in P_{ij}$ . Entonces, el subcamino de  $P_{ij}$  entre  $i$  y  $k$  es un camino mínimo entre  $i$  y  $k$ .

# Algoritmo de Dijkstra (1959)

---



Edsger Dijkstra (1930–2002)

[www.cs.utexas.edu/users/EWD](http://www.cs.utexas.edu/users/EWD)

# Algoritmo de Dijkstra (1959)

---

Asumimos que las longitudes de las aristas son positivas. El grafo puede ser orientado o no orientado.

1. Asignar **distancias tentativas**  $d_s = 0$  y  $d_i = \infty$  para  $i \neq s$ .

## Algoritmo de Dijkstra (1959)

---

Asumimos que las longitudes de las aristas son positivas. El grafo puede ser orientado o no orientado.

1. Asignar **distancias tentativas**  $d_s = 0$  y  $d_i = \infty$  para  $i \neq s$ .
2. Mientras el destino no esté visitado:

## Algoritmo de Dijkstra (1959)

---

Asumimos que las longitudes de las aristas son positivas. El grafo puede ser orientado o no orientado.

1. Asignar **distancias tentativas**  $d_s = 0$  y  $d_i = \infty$  para  $i \neq s$ .
2. Mientras el destino no esté visitado:
  - Seleccionar como nodo actual  $i$  el nodo no visitado con menor distancia tentativa, y marcarlo como visitado.

# Algoritmo de Dijkstra (1959)

---

Asumimos que las longitudes de las aristas son positivas. El grafo puede ser orientado o no orientado.

1. Asignar **distancias tentativas**  $d_s = 0$  y  $d_i = \infty$  para  $i \neq s$ .
2. Mientras el destino no esté visitado:
  - Seleccionar como nodo actual  $i$  el nodo no visitado con menor distancia tentativa, y marcarlo como visitado.
  - Para cada  $j \in N^+(i)$  no visitado, calcular  $d'_j = d_i + d_{ij}$ . Si  $d'_j < d_j$  entonces fijar  $d_j := d'_j$ .

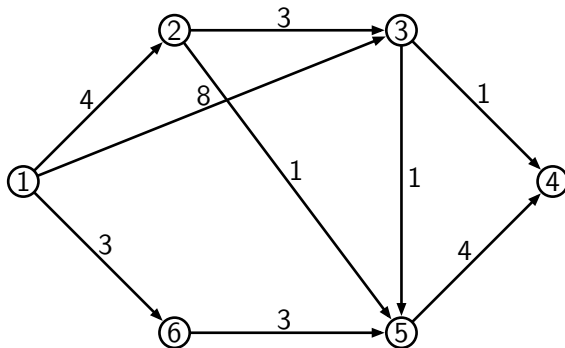
# Algoritmo de Dijkstra (1959)

---

Asumimos que las longitudes de las aristas son positivas. El grafo puede ser orientado o no orientado.

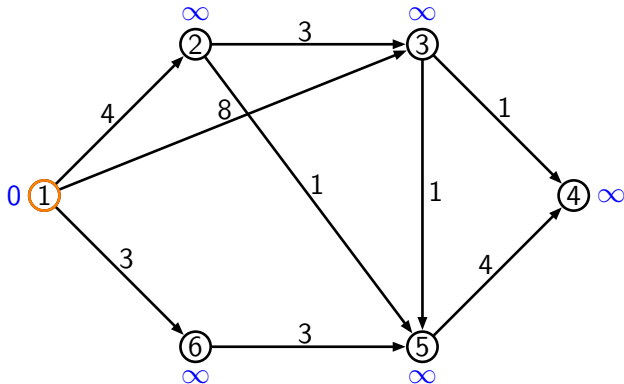
1. Asignar **distancias tentativas**  $d_s = 0$  y  $d_i = \infty$  para  $i \neq s$ .
2. Mientras el destino no esté visitado:
  - Seleccionar como nodo actual  $i$  el nodo no visitado con menor distancia tentativa, y marcarlo como visitado.
  - Para cada  $j \in N^+(i)$  no visitado, calcular  $d'_j = d_i + d_{ij}$ . Si  $d'_j < d_j$  entonces fijar  $d_j := d'_j$ .
3. Retornar  $d_t$ .

## Algoritmo de Dijkstra - Ejemplo



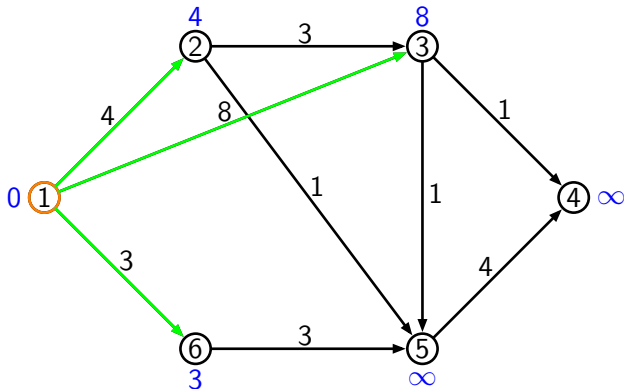
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1\}$$



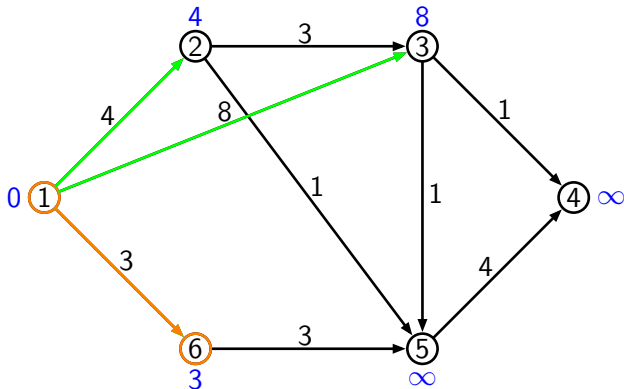
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1\}$$



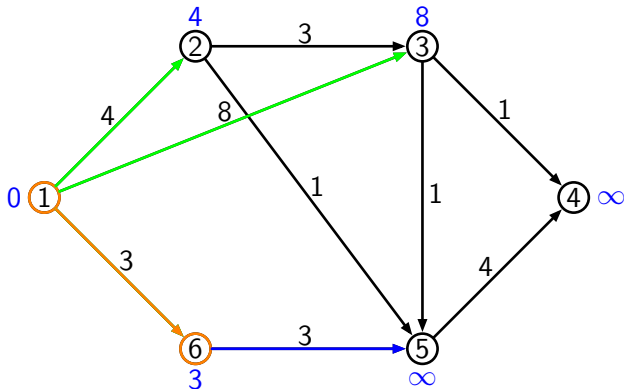
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6\}$$



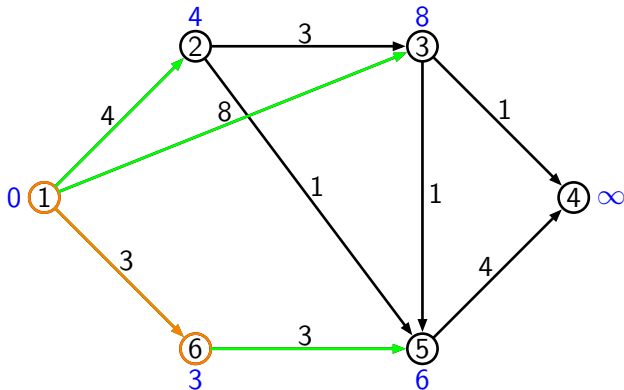
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6\}$$



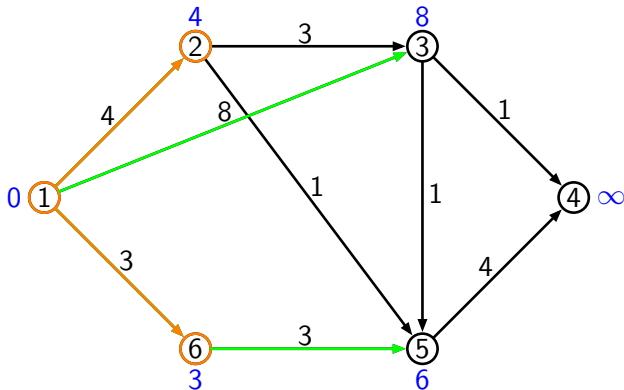
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6\}$$



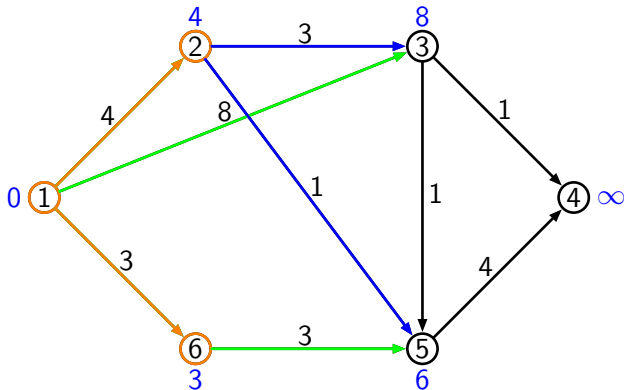
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6, 2\}$$



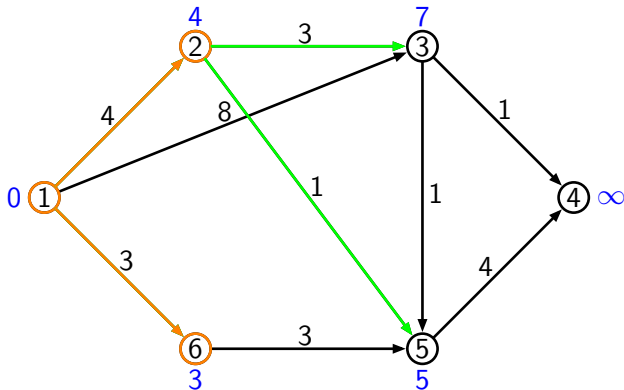
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6, 2\}$$



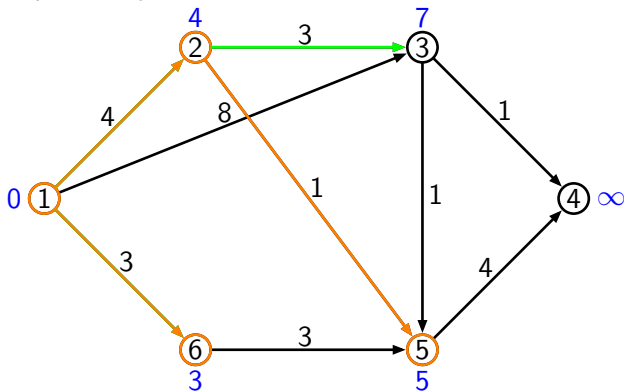
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6, 2\}$$



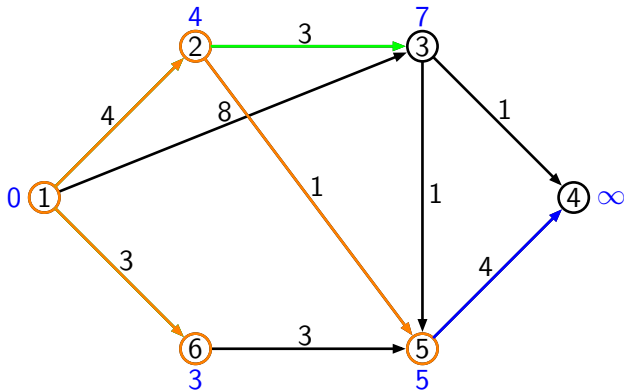
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6, 2, 5\}$$



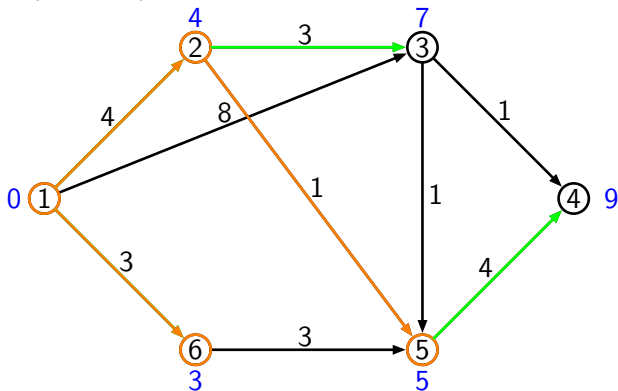
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6, 2, 5\}$$



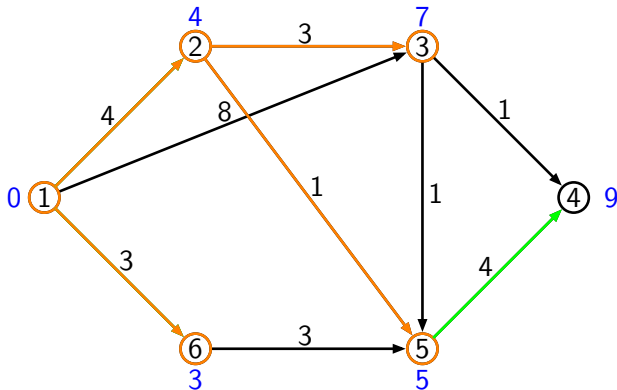
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6, 2, 5\}$$



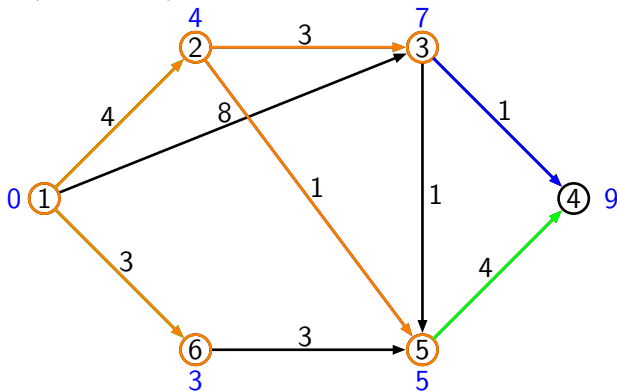
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6, 2, 5, 3\}$$



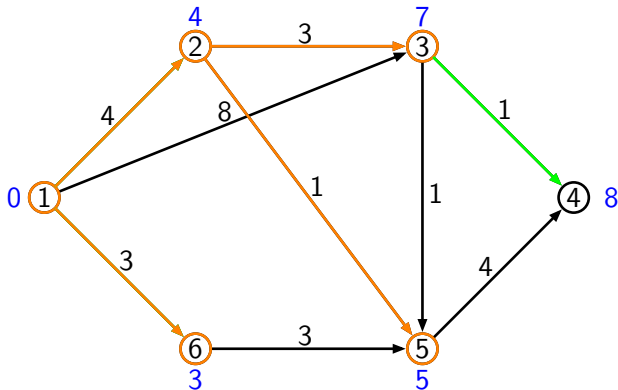
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6, 2, 5, 3\}$$



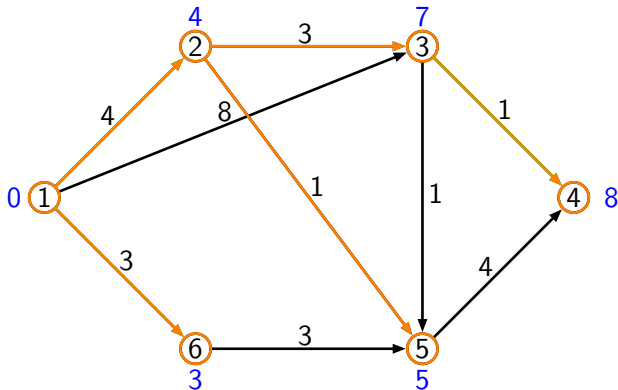
## Algoritmo de Dijkstra - Ejemplo

$$S = \{1, 6, 2, 5, 3\}$$



## Algoritmo de Dijkstra - Ejemplo

$S = \{1, 6, 2, 5, 3, 4\}$



# Algoritmo de Dijkstra

---

- Llamamos  $S_k$  a los nodos visitados luego de la iteración  $k$ .

# Algoritmo de Dijkstra

---

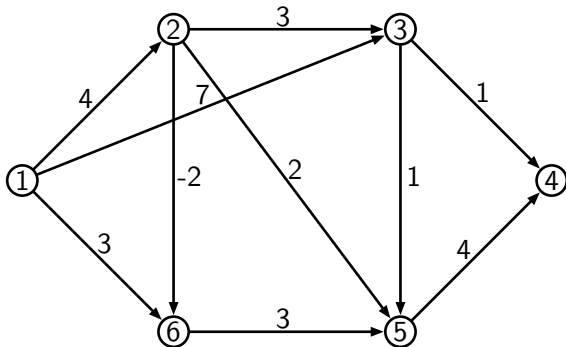
- Llamamos  $S_k$  a los nodos visitados luego de la iteración  $k$ .
- **Lema.** Al finalizar la iteración  $k$  el algoritmo de Dijkstra determina el camino mínimo entre el nodo  $s$  y cada uno de los nodos de  $S_k$ .

# Algoritmo de Dijkstra

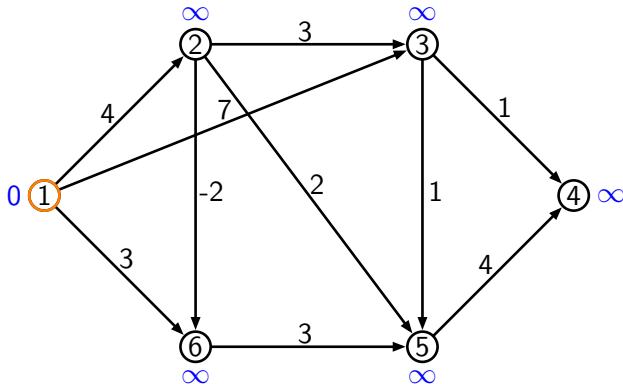
---

- Llamamos  $S_k$  a los nodos visitados luego de la iteración  $k$ .
- **Lema.** Al finalizar la iteración  $k$  el algoritmo de Dijkstra determina el camino mínimo entre el nodo  $s$  y cada uno de los nodos de  $S_k$ .
- **Teorema.** Dado un grafo orientado  $G$  con pesos positivos en las aristas, el algoritmo de Dijkstra determina el camino mínimo entre el nodo  $s$  y el resto de los nodos de  $G$ .

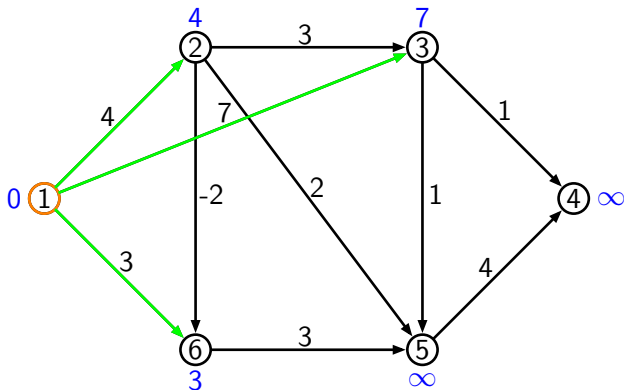
# Algoritmo de Dijkstra con pesos negativos



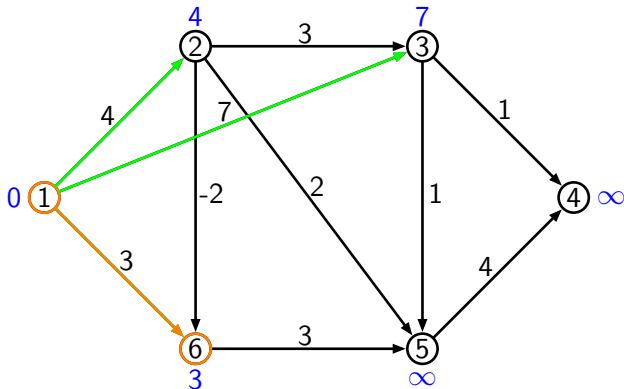
# Algoritmo de Dijkstra con pesos negativos



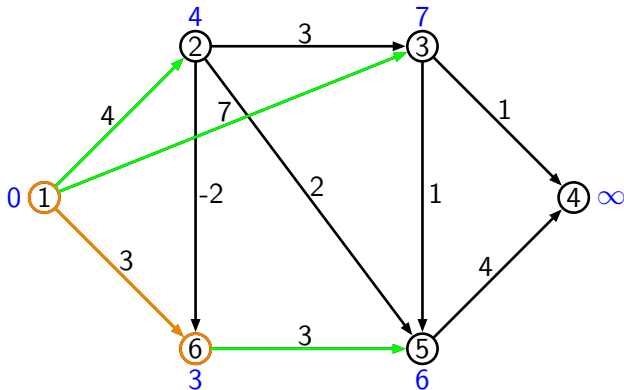
## Algoritmo de Dijkstra con pesos negativos



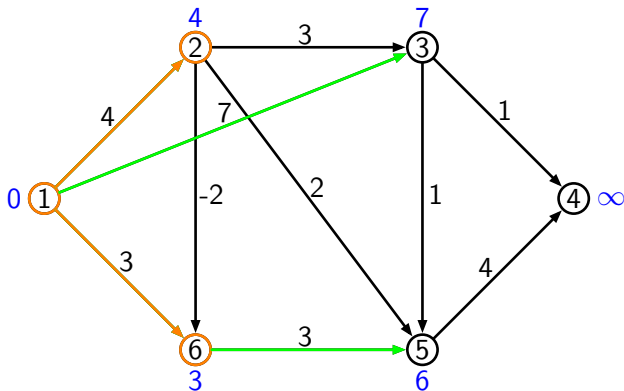
## Algoritmo de Dijkstra con pesos negativos



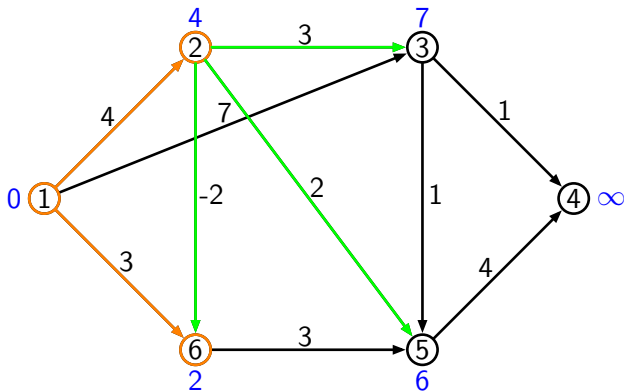
## Algoritmo de Dijkstra con pesos negativos



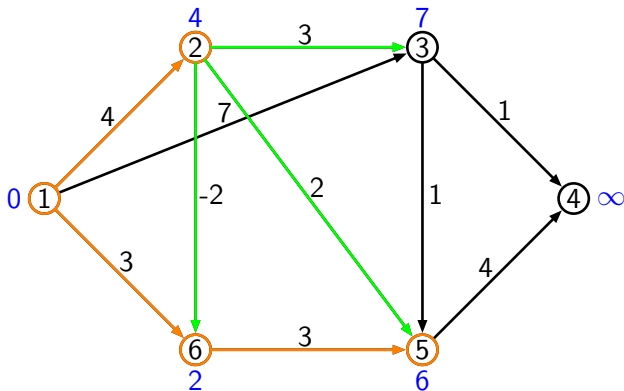
## Algoritmo de Dijkstra con pesos negativos



# Algoritmo de Dijkstra con pesos negativos



# Algoritmo de Dijkstra con pesos negativos



# Algoritmo de Dijkstra, complejidad computacional

---

1. Asignar **distancias tentativas**  $d_s = 0$  y  $d_i = \infty$  para  $i \neq s$ .
2. Mientras el destino no esté visitado:
  - Seleccionar como nodo actual  $i$  el nodo no visitado con menor distancia tentativa, y marcarlo como visitado.
  - Para cada  $j \in N^+(i)$  no visitado, calcular  $d'_j = d_i + d_{ij}$ . Si  $d'_j < d_j$  entonces fijar  $d_j := d'_j$ .
3. Retornar  $d_t$ .

# Algoritmo de Dijkstra, complejidad computacional

---

1. Asignar **distancias tentativas**  $d_s = 0$  y  $d_i = \infty$  para  $i \neq s$ .
2. Mientras el destino no esté visitado:
  - Seleccionar como nodo actual  $i$  el nodo no visitado con menor distancia tentativa, y marcarlo como visitado.
  - Para cada  $j \in N^+(i)$  no visitado, calcular  $d'_j = d_i + d_{ij}$ . Si  $d'_j < d_j$  entonces fijar  $d_j := d'_j$ .
3. Retornar  $d_t$ .

Complejidad computacional:

# Algoritmo de Dijkstra, complejidad computacional

1. Asignar **distancias tentativas**  $d_s = 0$  y  $d_i = \infty$  para  $i \neq s$ .
2. Mientras el destino no esté visitado:
  - Seleccionar como nodo actual  $i$  el nodo no visitado con menor distancia tentativa, y marcarlo como visitado.
  - Para cada  $j \in N^+(i)$  no visitado, calcular  $d'_j = d_i + d_{ij}$ . Si  $d'_j < d_j$  entonces fijar  $d_j := d'_j$ .
3. Retornar  $d_t$ .

Complejidad computacional:

- $O(n^2)$  con una implementación sencilla.

# Algoritmo de Dijkstra, complejidad computacional

1. Asignar **distancias tentativas**  $d_s = 0$  y  $d_i = \infty$  para  $i \neq s$ .
2. Mientras el destino no esté visitado:
  - Seleccionar como nodo actual  $i$  el nodo no visitado con menor distancia tentativa, y marcarlo como visitado.
  - Para cada  $j \in N^+(i)$  no visitado, calcular  $d'_j = d_i + d_{ij}$ . Si  $d'_j < d_j$  entonces fijar  $d_j := d'_j$ .
3. Retornar  $d_t$ .

Complejidad computacional:

- $O(n^2)$  con una implementación sencilla.
- $O(m \log n)$  guardando las distancias tentativas en un **heap**.



# Algoritmo de Dijkstra





# Algoritmo de Dijkstra bidireccional



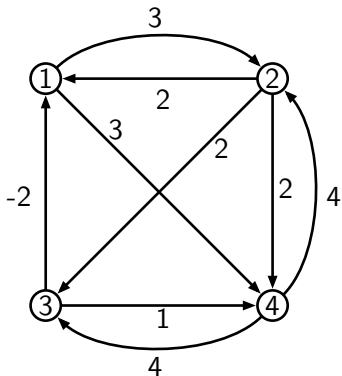
# Algoritmos matriciales

Sea  $G = (N, A)$  un digrafo y  $d : A \rightarrow R$  una función de distancia para los arcos de  $G$ . Definimos las matrices  $L = (l_{ij}) \in \mathbb{R}^{n \times n}$  y  $M = (m_{ij}) \in \mathbb{R}^{n \times n}$  como:

$$l_{ij} = \begin{cases} 0 & \text{si } i = j \\ d_{ij} & \text{si } ij \in A \\ \infty & \text{si } ij \notin A \end{cases}$$

$$m_{ij} = \begin{cases} \text{longitud del camino mínimo de } i \text{ a } j & \text{si existe alguno} \\ \infty & \text{si no} \end{cases}$$

# Algoritmos matriciales



$L =$

|   | 1        | 2        | 3        | 4 |
|---|----------|----------|----------|---|
| 1 | 0        | 3        | $\infty$ | 3 |
| 2 | 2        | 0        | 2        | 2 |
| 3 | -2       | $\infty$ | 0        | 1 |
| 4 | $\infty$ | 4        | 4        | 0 |

# Algoritmos matriciales

---

- A partir de la matriz  $L$ , podemos plantear el siguiente **algoritmo general**:
  1. Asignar  $M := L$ .
  2. Mientras existan nodos  $i, j$  y  $k$  con  $m_{ij} > m_{ik} + m_{kj}$ :
    - Asignar  $m_{ij} := m_{ik} + m_{kj}$ .
  3. retornar  $M$ .

# Algoritmos matriciales

---

- A partir de la matriz  $L$ , podemos plantear el siguiente **algoritmo general**:
  1. Asignar  $M := L$ .
  2. Mientras existan nodos  $i, j$  y  $k$  con  $m_{ij} > m_{ik} + m_{kj}$ :
    - Asignar  $m_{ij} := m_{ik} + m_{kj}$ .
  3. retornar  $M$ .
- ¿Por qué es correcto el algoritmo?

# Algoritmos matriciales

- A partir de la matriz  $L$ , podemos plantear el siguiente **algoritmo general**:
  1. Asignar  $M := L$ .
  2. Mientras existan nodos  $i, j$  y  $k$  con  $m_{ij} > m_{ik} + m_{kj}$ :
    - Asignar  $m_{ij} := m_{ik} + m_{kj}$ .
  3. retornar  $M$ .
- ¿Por qué es correcto el algoritmo?
- ¿Termina este algoritmo?

# Algoritmos matriciales

- A partir de la matriz  $L$ , podemos plantear el siguiente **algoritmo general**:
  1. Asignar  $M := L$ .
  2. Mientras existan nodos  $i, j$  y  $k$  con  $m_{ij} > m_{ik} + m_{kj}$ :
    - Asignar  $m_{ij} := m_{ik} + m_{kj}$ .
  3. retornar  $M$ .
- ¿Por qué es correcto el algoritmo?
- ¿Termina este algoritmo?
- ¿Cuál es su complejidad computacional?

## Algoritmo de Floyd-Warshall (1962)

---



Robert Floyd  
(1936–2001)



Stephen Warshall  
(1935–2006)

## Algoritmo de Floyd-Warshall (1962)

---

- A partir de la matriz  $L$  de distancias en el grafo, calcula la matriz  $M$  de distancias entre todos los nodos del grafo.

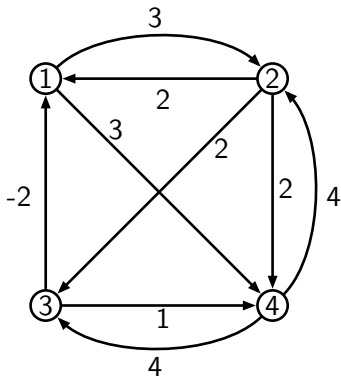
---

```

1  M := L;
2  for k := 1 to n do
3      for i := 1 to n do
4          for j := 1 to n do
5              {
6                  if( m[i,j] > m[i,k] + m[k,j] )
7                      m[i,j] := m[i,k] + m[k,j];
8              }
9
10 return M;
```

---

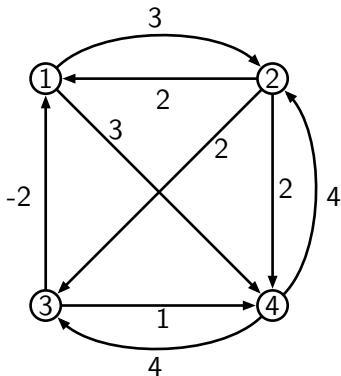
## Algoritmo de Floyd-Warshall (1962)



$L^0 =$

|   | 1        | 2        | 3        | 4 |
|---|----------|----------|----------|---|
| 1 | 0        | 3        | $\infty$ | 3 |
| 2 | 2        | 0        | 2        | 2 |
| 3 | -2       | $\infty$ | 0        | 1 |
| 4 | $\infty$ | 4        | 4        | 0 |

# Algoritmo de Floyd-Warshall (1962)



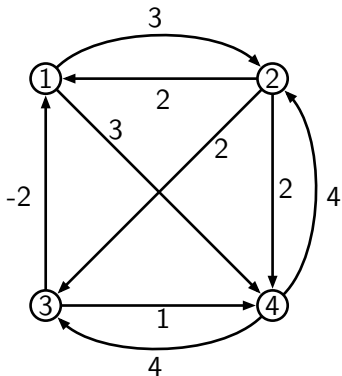
$L^0 =$

|   | 1        | 2        | 3        | 4 |
|---|----------|----------|----------|---|
| 1 | 0        | 3        | $\infty$ | 3 |
| 2 | 2        | 0        | 2        | 2 |
| 3 | -2       | $\infty$ | 0        | 1 |
| 4 | $\infty$ | 4        | 4        | 0 |

$L^1 =$

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 |   |   |   |   |
| 2 |   |   |   |   |
| 3 |   |   |   |   |
| 4 |   |   |   |   |

## Algoritmo de Floyd-Warshall (1962)



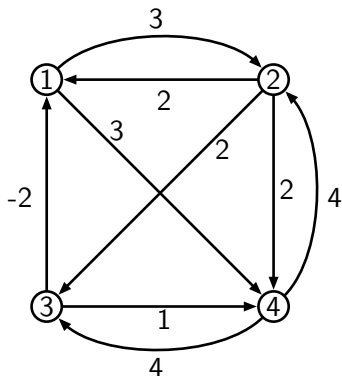
$L^0 =$

|   | 1        | 2        | 3        | 4 |
|---|----------|----------|----------|---|
| 1 | 0        | 3        | $\infty$ | 3 |
| 2 | 2        | 0        | 2        | 2 |
| 3 | -2       | $\infty$ | 0        | 1 |
| 4 | $\infty$ | 4        | 4        | 0 |

$L^1 =$

|   | 1  | 2 | 3        | 4 |
|---|----|---|----------|---|
| 1 | 0  | 3 | $\infty$ | 3 |
| 2 | 2  | 0 | 2        | 2 |
| 3 | -2 |   |          |   |
| 4 |    |   |          |   |

# Algoritmo de Floyd-Warshall (1962)



$L^0 =$

|   | 1        | 2        | 3        | 4 |
|---|----------|----------|----------|---|
| 1 | 0        | 3        | $\infty$ | 3 |
| 2 | 2        | 0        | 2        | 2 |
| 3 | -2       | $\infty$ | 0        | 1 |
| 4 | $\infty$ | 4        | 4        | 0 |

$L^1 =$

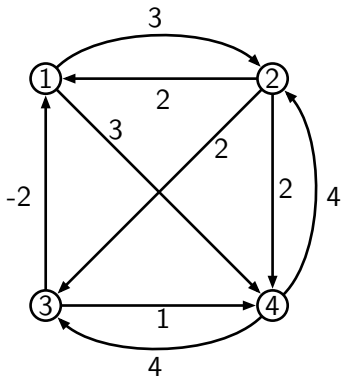
|   | 1  | 2 | 3        | 4 |
|---|----|---|----------|---|
| 1 | 0  | 3 | $\infty$ | 3 |
| 2 | 2  | 0 | 2        | 2 |
| 3 | -2 |   |          |   |
| 4 |    |   |          |   |

23 of 32  $k=1$

$i=3$

$j=2$

# Algoritmo de Floyd-Warshall (1962)



$L^0 =$

|   | 1        | 2        | 3        | 4 |
|---|----------|----------|----------|---|
| 1 | 0        | 3        | $\infty$ | 3 |
| 2 | 2        | 0        | 2        | 2 |
| 3 | -2       | $\infty$ | 0        | 1 |
| 4 | $\infty$ | 4        | 4        | 0 |

$L^1 =$

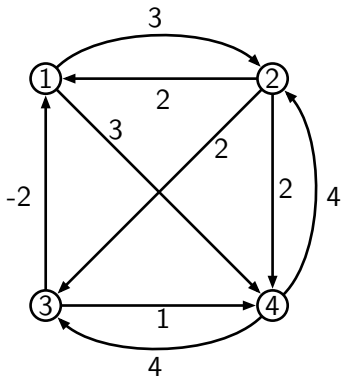
|   | 1  | 2 | 3        | 4 |
|---|----|---|----------|---|
| 1 | 0  | 3 | $\infty$ | 3 |
| 2 | 2  | 0 | 2        | 2 |
| 3 | -2 | 1 |          |   |
| 4 |    |   |          |   |

23 of 32  $k=1$

$i=3$

$j=2$

## Algoritmo de Floyd-Warshall (1962)



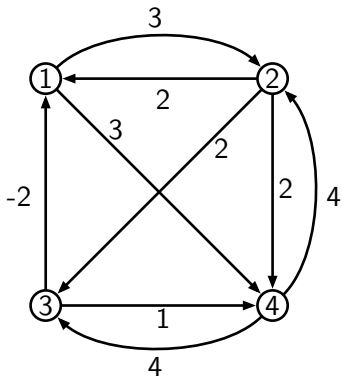
$L^0 =$

|   | 1        | 2        | 3        | 4 |
|---|----------|----------|----------|---|
| 1 | 0        | 3        | $\infty$ | 3 |
| 2 | 2        | 0        | 2        | 2 |
| 3 | -2       | $\infty$ | 0        | 1 |
| 4 | $\infty$ | 4        | 4        | 0 |

$L^1 =$

|   | 1        | 2 | 3        | 4 |
|---|----------|---|----------|---|
| 1 | 0        | 3 | $\infty$ | 3 |
| 2 | 2        | 0 | 2        | 2 |
| 3 | -2       | 1 | 0        | 1 |
| 4 | $\infty$ | 4 | 4        | 0 |

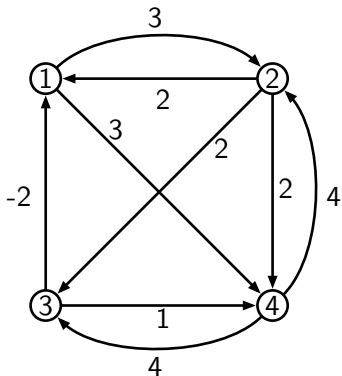
## Algoritmo de Floyd-Warshall (1962)



$L^1 =$

|   | 1        | 2 | 3        | 4 |
|---|----------|---|----------|---|
| 1 | 0        | 3 | $\infty$ | 3 |
| 2 | 2        | 0 | 2        | 2 |
| 3 | -2       | 1 | 0        | 1 |
| 4 | $\infty$ | 4 | 4        | 0 |

# Algoritmo de Floyd-Warshall (1962)



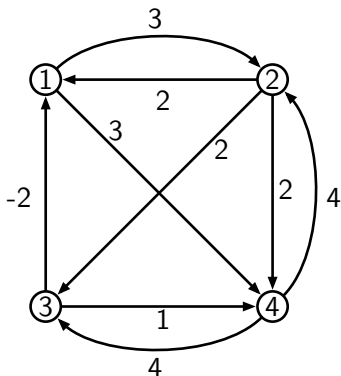
$L^1 =$

|   | 1        | 2 | 3        | 4 |
|---|----------|---|----------|---|
| 1 | 0        | 3 | $\infty$ | 3 |
| 2 | 2        | 0 | 2        | 2 |
| 3 | -2       | 1 | 0        | 1 |
| 4 | $\infty$ | 4 | 4        | 0 |

$L^2 =$

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 |   |   |
| 2 |   |   |   |   |
| 3 |   |   |   |   |
| 4 |   |   |   |   |

# Algoritmo de Floyd-Warshall (1962)



$L^1 =$

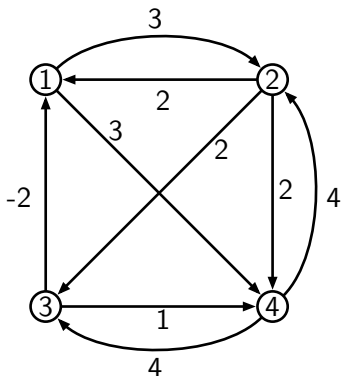
|   | 1        | 2 | 3        | 4 |
|---|----------|---|----------|---|
| 1 | 0        | 3 | $\infty$ | 3 |
| 2 | 2        | 0 | 2        | 2 |
| 3 | -2       | 1 | 0        | 1 |
| 4 | $\infty$ | 4 | 4        | 0 |

$L^2 =$

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 |   |   |
| 2 |   |   |   |   |
| 3 |   |   |   |   |
| 4 |   |   |   |   |

23 of 32  $k=2$   $i=1$   $j=3$

# Algoritmo de Floyd-Warshall (1962)



$L^1 =$

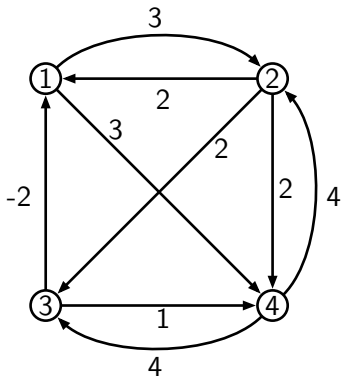
|   | 1        | 2 | 3        | 4 |
|---|----------|---|----------|---|
| 1 | 0        | 3 | $\infty$ | 3 |
| 2 | 2        | 0 | 2        | 2 |
| 3 | -2       | 1 | 0        | 1 |
| 4 | $\infty$ | 4 | 4        | 0 |

$L^2 =$

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 5 |   |
| 2 |   |   |   |   |
| 3 |   |   |   |   |
| 4 |   |   |   |   |

23 of 32  $k=2$   $i=1$   $j=3$

# Algoritmo de Floyd-Warshall (1962)

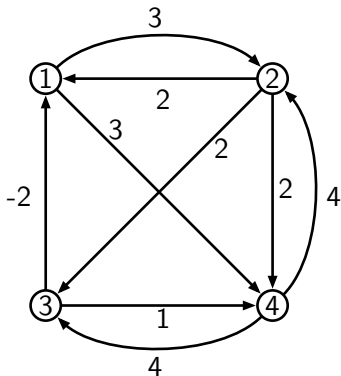

 $L^1 =$ 

|   | 1        | 2 | 3        | 4 |
|---|----------|---|----------|---|
| 1 | 0        | 3 | $\infty$ | 3 |
| 2 | 2        | 0 | 2        | 2 |
| 3 | -2       | 1 | 0        | 1 |
| 4 | $\infty$ | 4 | 4        | 0 |

 $L^2 =$ 

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 |    |   |   |   |

# Algoritmo de Floyd-Warshall (1962)


 $L^1 =$ 

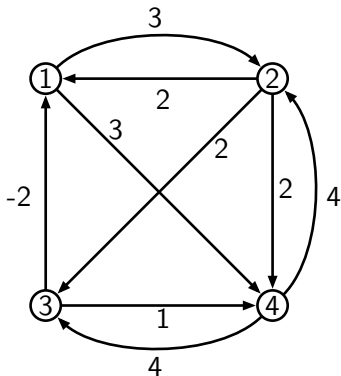
|   | 1        | 2 | 3        | 4 |
|---|----------|---|----------|---|
| 1 | 0        | 3 | $\infty$ | 3 |
| 2 | 2        | 0 | 2        | 2 |
| 3 | -2       | 1 | 0        | 1 |
| 4 | $\infty$ | 4 | 4        | 0 |

 $L^2 =$ 

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 |    |   |   |   |

 $k=2$ 
 $i=4$ 
 $j=1$

# Algoritmo de Floyd-Warshall (1962)


 $L^1 =$ 

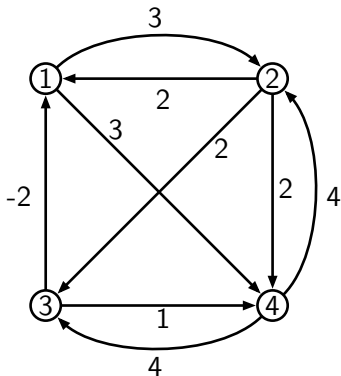
|   | 1        | 2 | 3        | 4 |
|---|----------|---|----------|---|
| 1 | 0        | 3 | $\infty$ | 3 |
| 2 | 2        | 0 | 2        | 2 |
| 3 | -2       | 1 | 0        | 1 |
| 4 | $\infty$ | 4 | 4        | 0 |

 $L^2 =$ 

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 6  |   |   |   |

 $k=2$ 
 $i=4$ 
 $j=1$

# Algoritmo de Floyd-Warshall (1962)



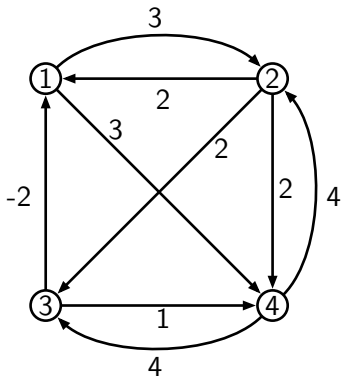
$L^1 =$

|   | 1        | 2 | 3        | 4 |
|---|----------|---|----------|---|
| 1 | 0        | 3 | $\infty$ | 3 |
| 2 | 2        | 0 | 2        | 2 |
| 3 | -2       | 1 | 0        | 1 |
| 4 | $\infty$ | 4 | 4        | 0 |

$L^2 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 6  | 4 | 4 | 0 |

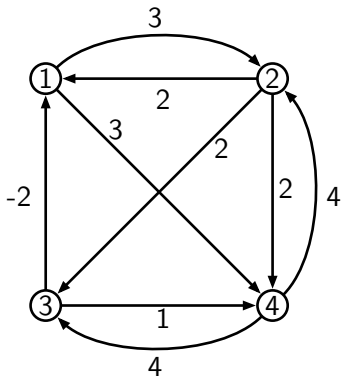
## Algoritmo de Floyd-Warshall (1962)



$L^2 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 6  | 4 | 4 | 0 |

## Algoritmo de Floyd-Warshall (1962)



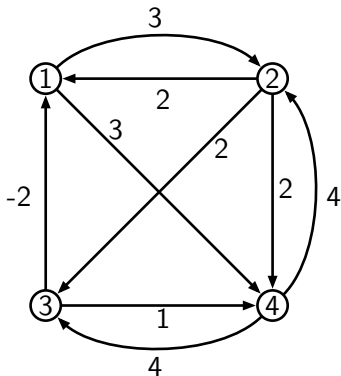
$L^2 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 6  | 4 | 4 | 0 |

$L^3 =$

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 5 | 3 |
| 2 |   |   |   |   |
| 3 |   |   |   |   |
| 4 |   |   |   |   |

# Algoritmo de Floyd-Warshall (1962)



$L^2 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 6  | 4 | 4 | 0 |

$L^3 =$

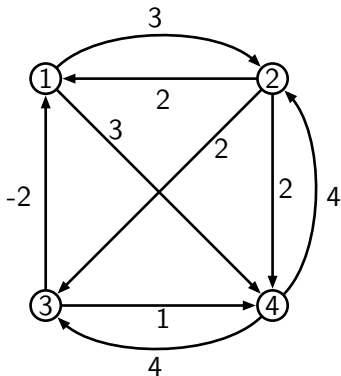
|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 5 | 3 |
| 2 |   |   |   |   |
| 3 |   |   |   |   |
| 4 |   |   |   |   |

$k=3$

$i=2$

$j=1$

# Algoritmo de Floyd-Warshall (1962)



$L^2 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 6  | 4 | 4 | 0 |

$L^3 =$

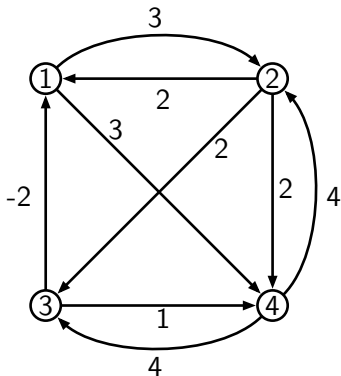
|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 5 | 3 |
| 2 | 0 |   |   |   |
| 3 |   |   |   |   |
| 4 |   |   |   |   |

$k=3$

$i=2$

$j=1$

## Algoritmo de Floyd-Warshall (1962)



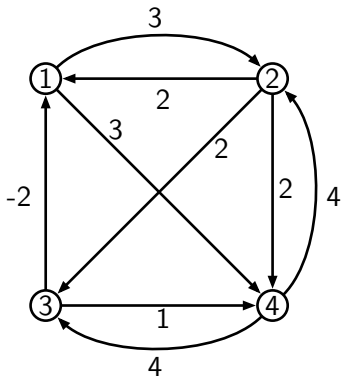
$L^2 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 6  | 4 | 4 | 0 |

$L^3 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 0  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 |    |   |   |   |

# Algoritmo de Floyd-Warshall (1962)



$L^2 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 6  | 4 | 4 | 0 |

$L^3 =$

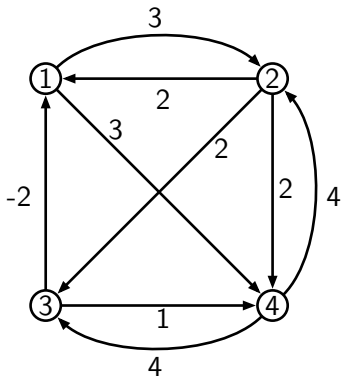
|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 0  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 |    |   |   |   |

$k=3$

$i=4$

$j=1$

# Algoritmo de Floyd-Warshall (1962)



$L^2 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 6  | 4 | 4 | 0 |

$L^3 =$

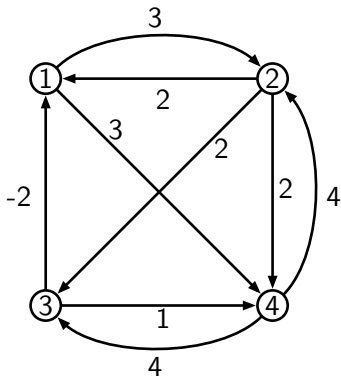
|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 0  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 2  |   |   |   |

23 of 32  $k=3$

$i=4$

$j=1$

## Algoritmo de Floyd-Warshall (1962)



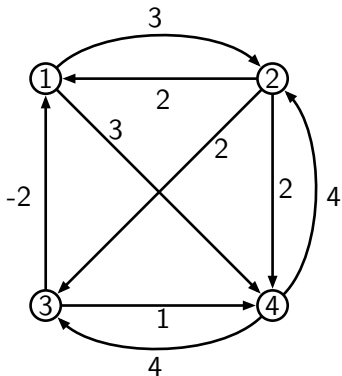
$L^2 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 2  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 6  | 4 | 4 | 0 |

$L^3 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 0  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 2  | 4 | 4 | 0 |

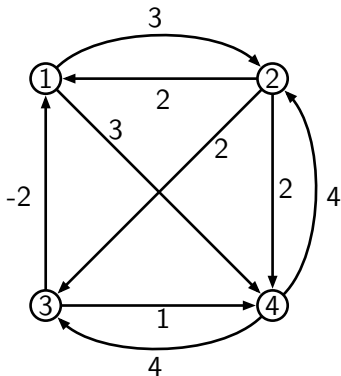
## Algoritmo de Floyd-Warshall (1962)



$L^3 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 0  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 2  | 4 | 4 | 0 |

# Algoritmo de Floyd-Warshall (1962)



$L^3 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 0  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 2  | 4 | 4 | 0 |

$D = L^4 =$

|   | 1  | 2 | 3 | 4 |
|---|----|---|---|---|
| 1 | 0  | 3 | 5 | 3 |
| 2 | 0  | 0 | 2 | 2 |
| 3 | -2 | 1 | 0 | 1 |
| 4 | 2  | 4 | 4 | 0 |

# Algoritmo de Floyd-Warshall (1962)

---

- **Teorema.** El algoritmo de Floyd-Warshall determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin circuitos negativos.

# Algoritmo de Floyd-Warshall (1962)

---

- **Teorema.** El algoritmo de Floyd-Warshall determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin circuitos negativos.
1. ¿Cuál es la complejidad de algoritmo de Floyd-Warshall?

# Algoritmo de Floyd-Warshall (1962)

---

- **Teorema.** El algoritmo de Floyd-Warshall determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin circuitos negativos.
  1. ¿Cuál es la complejidad de algoritmo de Floyd-Warshall?
  2. ¿Cuánta memoria requiere?

# Algoritmo de Floyd-Warshall (1962)

---

- **Teorema.** El algoritmo de Floyd-Warshall determina los caminos mínimos entre todos los pares de nodos de un grafo orientado sin circuitos negativos.
  1. ¿Cuál es la complejidad de algoritmo de Floyd-Warshall?
  2. ¿Cuánta memoria requiere?
  3. ¿Cómo se puede adaptar para detectar si el grafo tiene circuitos de longitud negativa?

# Algoritmo de Floyd-Warshall (1962)

---

```

1  M := L;
2  for k := 1 to n do
3      for i := 1 to n do
4          {
5              if(  $m[i,k] + m[k,i] < 0$  )
6                  return "Hay ciclos negativos!";
7
8              for j := 1 to n do
9                  {
10                     if(  $m[i,j] > m[i,k] + m[k,j]$  )
11                          $m[i,j] := m[i,k] + m[k,j]$ ;
12                 }
13         }
14
15 return M;
```

---

## Reducciones desde otros problemas

---

- **Exact packing.** Dados un conjunto  $C = \{1, \dots, n\}$  de objetos, el peso  $p_i \in \mathbb{Z}_+$  de cada objeto  $i \in C$  y una capacidad máxima  $M \in \mathbb{Z}_+$ , determinar si existe un subconjunto de  $C$  con peso igual a  $M$ .

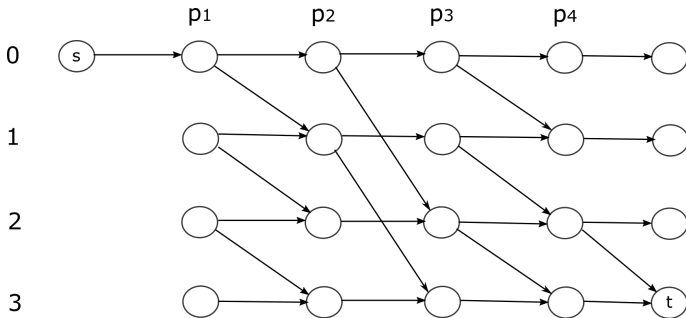
## Reducciones desde otros problemas

- **Exact packing.** Dados un conjunto  $C = \{1, \dots, n\}$  de objetos, el peso  $p_i \in \mathbb{Z}_+$  de cada objeto  $i \in C$  y una capacidad máxima  $M \in \mathbb{Z}_+$ , determinar si existe un subconjunto de  $C$  con peso igual a  $M$ .
- Ejemplo:
  1.  $C = \{1, \dots, 4\}$ ,
  2.  $p_1 = 1, p_2 = 2, p_3 = 1, p_4 = 1$ ,
  3.  $M = 3$ .

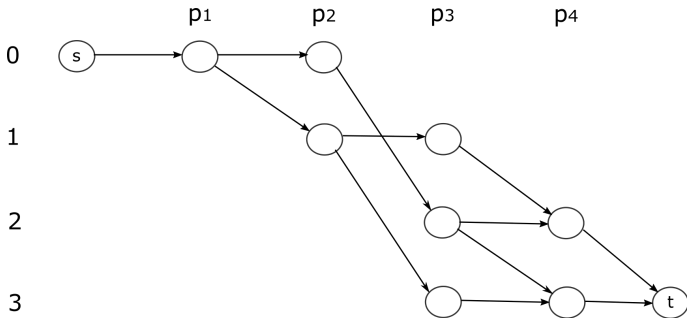
## Reducciones desde otros problemas

- **Exact packing.** Dados un conjunto  $C = \{1, \dots, n\}$  de objetos, el peso  $p_i \in \mathbb{Z}_+$  de cada objeto  $i \in C$  y una capacidad máxima  $M \in \mathbb{Z}_+$ , determinar si existe un subconjunto de  $C$  con peso igual a  $M$ .
- Ejemplo:
  1.  $C = \{1, \dots, 4\}$ ,
  2.  $p_1 = 1, p_2 = 2, p_3 = 1, p_4 = 1$ ,
  3.  $M = 3$ .
- ¿Cómo convertimos este problema en un problema de recorrido en un grafo?

# Reducciones desde otros problemas

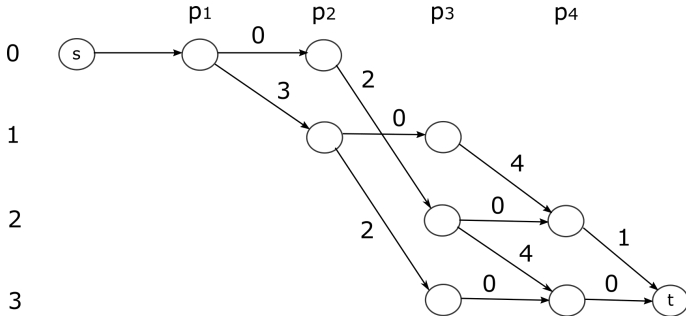


# Reducciones desde otros problemas



# Reducciones desde otros problemas

Si además cada objeto tiene un costo de compra y queremos minimizar el costo total ...



## Reducciones desde otros problemas

---

- **Course scheduling.** Dado el programa de charlas de una conferencia, determinar la mayor cantidad de charlas a la que podemos asistir.

## Reducciones desde otros problemas

---

- **Course scheduling.** Dado el programa de charlas de una conferencia, determinar la mayor cantidad de charlas a la que podemos asistir.
  1. Todas las charlas se realizan en un mismo día.
  2. Cada charla tiene un horario de inicio y final.
  3. Una vez que entramos a una charla, no se puede salir hasta que termine.

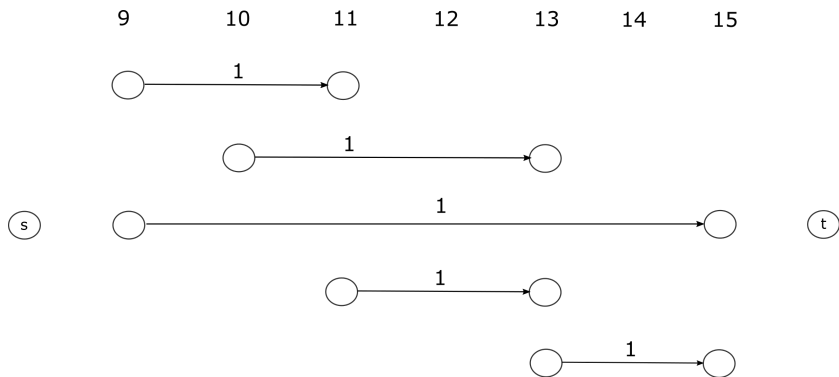
## Reducciones desde otros problemas

---

- **Course scheduling.** Dado el programa de charlas de una conferencia, determinar la mayor cantidad de charlas a la que podemos asistir.
  1. Todas las charlas se realizan en un mismo día.
  2. Cada charla tiene un horario de inicio y final.
  3. Una vez que entramos a una charla, no se puede salir hasta que termine.
- ¿Cómo convertimos este problema en un problema de camino mínimo?

## Reducciones desde otros problemas

**Ejemplo.** [9-11], [10-13], [9-15], [11-13], [13-15]



# Reducciones desde otros problemas

**Ejemplo.** [9-11], [10-13], [9-15], [11-13], [13-15]

